
天津大学

《智能搜索与问答》实验报告



Lab4 机器阅读理解系统（模型）的实现

学 号 3021244141

姓 名 于梦洁

学 院 智能与计算学部

专 业 计算机科学与技术

年 级 2021 级

班 级 2 班

2024 年 4 月 20 日

目录

一、 数据准备与处理	3
1.1 数据集	3
1.2 数据集预处理	3
二、 模型与算法介绍	4
2.1 模型结构	4
2.2 评估指标	5
三、 实验流程	6
1.conda 配置环境	6
2.训练 SAN 模型	7
四、 实验结果	8
五、 结果分析	10
六、 心得体会	11

一、 数据准备与处理

1.1 数据集

SQuAD，是一个阅读理解数据集，由众包工作者在一组维基百科文章上提出的问题组成，其中每个问题的答案都是相应文章中的一段文本，某些问题可能无法回答。本实验使用的数据集是 SQuAD_v1.1，它包含针对 500+ 文章的 10 万+ 问答对。下载 SQuAD_v1.1 的数据集，得到训练集 train-v1.1.json 和验证集 dev-v1.1.json。

```
{
  "answers": {
    "answer_start": [94, 87, 94, 94],
    "text": ["10th and 11th centuries", "in the 10th and 11th centuries", "10th and 11th centuries",
             "10th and 11th centuries"]
  },
  "context": "\"The Normans (Norman: Nourmands; French: Normands; Latin: Normanni) were the people who in the 10th and 11th centuries gave thei...",
  "id": "56dde6b9a695914005b9629",
  "question": "When were the Normans in Normandy?",
  "title": "Normans"
}
```

图 1. 数据格式

1.2 数据集预处理

数据需要进行预处理，构建问题和文章的不同特征。

```
def feature_func(sample, query_tokend, doc_tokend, vocab, vocab_tag, vocab_ner, is_train, v2_on=False):
    # features
    fea_dict = {}
    fea_dict['uid'] = sample['uid']
    if v2_on and is_train:
        fea_dict['label'] = sample['label']
    fea_dict['query_tok'] = tok_func(query_tokend, vocab)
    fea_dict['query_pos'] = postag_func(query_tokend, vocab_tag)
    fea_dict['query_ner'] = nertag_func(query_tokend, vocab_ner)
    fea_dict['doc_tok'] = tok_func(doc_tokend, vocab)
    fea_dict['doc_pos'] = postag_func(doc_tokend, vocab_tag)
    fea_dict['doc_ner'] = nertag_func(doc_tokend, vocab_ner)
    fea_dict['doc_fea'] = '{}'.format(match_func(query_tokend, doc_tokend))
    fea_dict['query_fea'] = '{}'.format(match_func(doc_tokend, query_tokend))
    doc_toks = [t.text for t in doc_tokend if len(t.text) > 0]
    query_toks = [t.text for t in query_tokend if len(t.text) > 0]
    answer_start = sample['answer_start']
    answer_end = sample['answer_end']
    answer = sample['answer']
    fea_dict['doc_ctok'] = doc_toks
    fea_dict['query_ctok'] = query_toks

    start, end, span = build_span(sample['context'], answer, doc_toks, answer_start,
                                   answer_end, is_train=is_train)
```

图 2. 数据预处理

二、 模型与算法介绍

2.1 模型结构

这个模型整个的框架上和主流的 NLI 模型都是差不多的,在 embedding 层采用的是 word embedding + char embedding。只经过了一个 stacked BiLSTM, 一个 attention 外加另一个 BiLSTM, 最后再用 GRU 完成了多步推理。其中, 这个多步推理就是 GRU 中的每个时间步, 在每个时间步上会有一个 softmax 操作, 然后得到了好多个 (时间步个数) 概率结果, 之后再求平均。整体模型结构见下图:

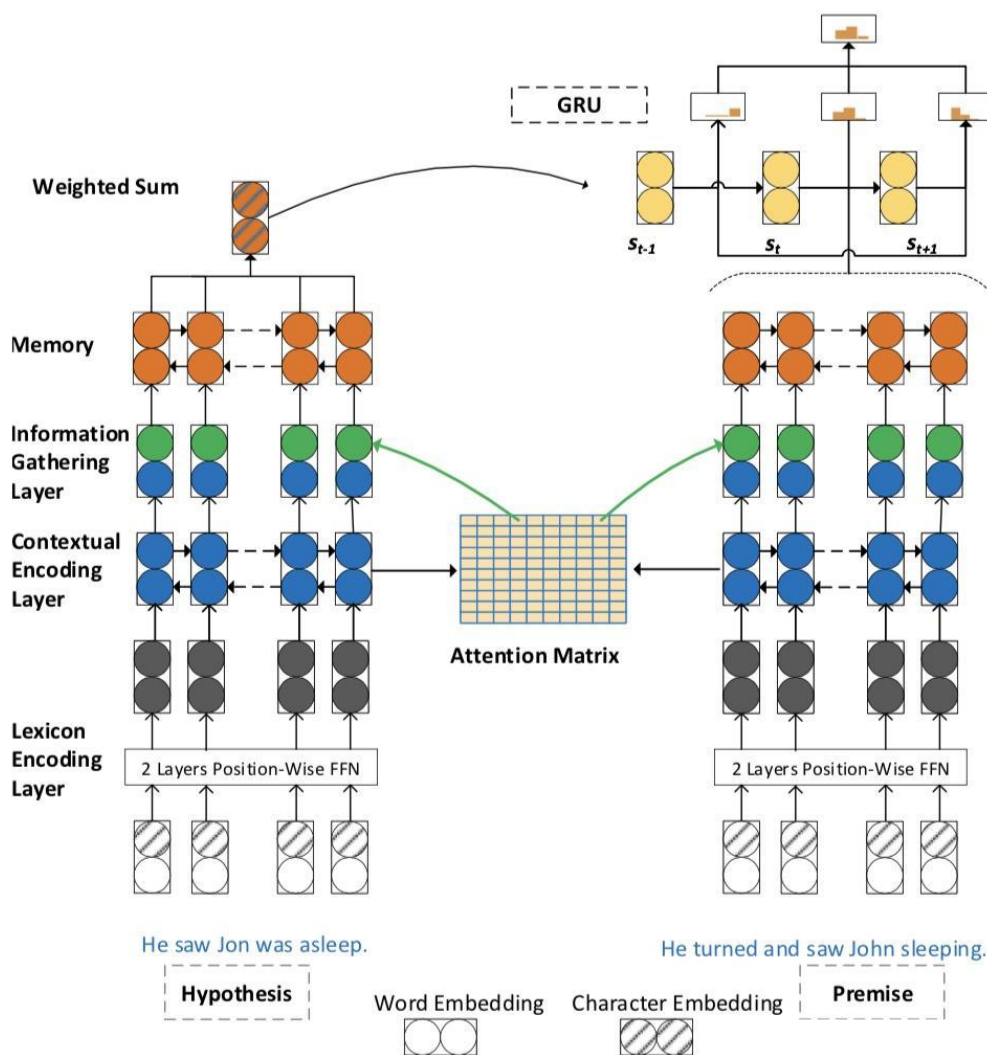


图 3. 模型整体架构

整个模型分为了四个层, 分别是: lexicon encoding layer, contextual encoding layer, memory generation layer, final answer model。

1. Lexicon encoding layer

这里就是 embedding 层, 采用了传统的 word embedding 和 char embedding 的

结合。两种 embedding 方式采用 concat 链接。然后经过了一个两层的前向网络完成降维。这里采用 feedforward network 的目的一个就是降维，另一个就是将不同 embedding 的信息进行整合，同时提升网络的表达能力。

2.Contextual Encoding Layer

在词库编码层上使用了两个堆叠的 BiLSTM 层来编码 P 和 H 中每个词的上下文信息。由于是双向层，它的隐藏层大小增加了一倍。在 BiLSTM 上使用一个 maxout 层，将其输出缩减到原来的隐藏层大小。

3.Memory Layer

这里的记忆层其实就是采用了 attention，通过注意力机制来构建工作记忆，采用点乘法注意力来衡量 P 和 H 中的 token 之间的相似性。

4.Answer module

答案模块将经过 T 个时间步的计算，并最终得到结果。

2.2 评估指标

该模型使用两个评估指标，分别为 EM 和 F1。

EM (Exact Match)，精确匹配指标，用于衡量模型在分类任务中的准确性。它表示模型的预测结果是否完全与真实结果匹配。如果预测结果与真实结果完全相同，则 EM 值为 100，否则为 0。EM 值越高，表示模型的预测结果与真实结果匹配得越好。

F1 Score，是精确率（Precision）和召回率（Recall）的调和平均值。精确率表示模型预测的正例中有多少是真正的正例，召回率表示真正的正例中有多少被模型正确地预测出来。F1 分数综合考虑了模型的准确性和完整性。F1 分数的取值范围在 0 到 100 之间，值越高表示模型的性能越好。

```
def evaluate(dataset, predictions):
    f1 = exact_match = total = 0
    for article in dataset:
        for paragraph in article['paragraphs']:
            for qa in paragraph['qas']:
                total += 1
                if qa['id'] not in predictions:
                    message = 'Unanswered question ' + qa['id'] + \
                        ' will receive score 0.'
                    print(message, file=sys.stderr)
                    continue
                ground_truths = list(map(lambda x: x['text'], qa['answers']))
                prediction = predictions[qa['id']]
                exact_match += metric_max_over_ground_truths(
                    exact_match_score, prediction, ground_truths)
                f1 += metric_max_over_ground_truths(
                    f1_score, prediction, ground_truths)

    exact_match = 100.0 * exact_match / total
    f1 = 100.0 * f1 / total
    return {'exact_match': exact_match, 'f1': f1}
```

图 4. 评估指标

三、 实验流程

1.conda 配置环境

使用命令“conda create -n san_mrc python=3.6”下载名为 san_mrc 的环境，其中 python 的版本为 3.6。运行命令“conda activate san_mrc”激活虚拟环境。

```
root@autodl-container-4fce119e52-1d7b1e9d:~/SAN-MRC# conda create -n san_mrc python=3.6
Collecting package metadata (current_repodata.json): done
Solving environment: done

==> WARNING: A newer version of conda exists. <==
current version: 4.10.3
latest version: 24.4.0

Please update conda by running

$ conda update -n base -c defaults conda

## Package Plan ##

environment location: /root/.miniconda3/envs/san_mrc

added / updated specs:
- python=3.6

The following packages will be downloaded:
```

package	build	size	url
_libgcc_mutex-0.1	main	3 KB	https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/main
_openmp_mutex-5.1	1_gnu	21 KB	https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/main
ca-certificates-2024.3.11	h06a4308_0	127 KB	https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/main
certifi-2020.6.20	pyhd3eb1b0_3	155 KB	https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/main
ld_impl_linux-64-2.38	h1181459_1	654 KB	https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/main
libffi-3.3	he6710b0_2	50 KB	https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/main
libgcc-ng-11.2.0	h1234567_1	5.3 MB	https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/main
libgomp-11.2.0	h1234567_1	474 KB	https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/main
libstdcxx-ng-11.2.0	h1234567_1	4.7 MB	https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/main
ncurses-6.4	h6a678d5_0	914 KB	https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/main

图 5. 创建虚拟环境

运行命令行“python train.py”训练模型。

四、实验结果

由于输出过多，这里只截取部分结果。

```
2024-05-16 11:18:05 [main      ] INFO      [Launching the SAN]
2024-05-16 11:18:05 [main      ] INFO      [Loading data]
2024-05-16 11:19:02 [main      ] INFO      [
##### Model Arch of SAN #####
DNetwork(
  (dropout): DropoutWrapper()
  (lexicon_encoder): LexiconEncoder(
    (dropout): DropoutWrapper()
    (dropout_emb): DropoutWrapper()
    (dropout_cove): DropoutWrapper()
    (embedding): Embedding(90981, 300, padding_idx=0)
    (ContextualEmbed): ContextualEmbed(
      (embedding): Embedding(90981, 300, padding_idx=0)
      (rnn1): LSTM(300, 300, bidirectional=True)
      (rnn2): LSTM(600, 300, bidirectional=True)
    )
  )
  (prealign): AttentionWrapper(
    (score_func): SimilarityWrapper(
      (score_func): DotProductProject(
        (dropout): DropoutWrapper()
        (proj_1): Linear(in_features=300, out_features=128, bias=False)
        (proj_2): Linear(in_features=300, out_features=128, bias=False)
      )
    )
  )
  (pos_embedding): Embedding(54, 12, padding_idx=0)
  (ner_embedding): Embedding(41, 8, padding_idx=0)
  (doc_pwnn): PositionwiseNN(
    (w_0): Conv1d(1224, 256, kernel_size=(1,), stride=(1,))
    (w_1): Conv1d(256, 256, kernel_size=(1,), stride=(1,))
    (dropout): DropoutWrapper()
  )
  (que_pwnn): PositionwiseNN(
    (w_0): Conv1d(900, 256, kernel_size=(1,), stride=(1,))
    (w_1): Conv1d(256, 256, kernel_size=(1,), stride=(1,))
    (dropout): DropoutWrapper()
  )
  )
  (doc_encoder_low): OneLayerBRNN(
    (dropout): DropoutWrapper()
    (rnn): LSTM(856, 128, bidirectional=True)
  )
  (doc_encoder_high): OneLayerBRNN(
    (dropout): DropoutWrapper()
    (rnn): LSTM(856, 128, bidirectional=True)
  )
  (query_encoder_low): OneLayerBRNN(
    (dropout): DropoutWrapper()
    (rnn): LSTM(856, 128, bidirectional=True)
  )
)
```

图 10. 模型训练结果示例 1


```

2024-05-16 11:19:35 [main ] WARNING [At epoch 0]
2024-05-16 11:19:36 [main ] INFO    [#updates[ 1] train loss[10.12298] remaining[0:35:56]]
2024-05-16 11:20:09 [main ] INFO    [#updates[ 100] train loss[8.75332] remaining[0:15:01]]
2024-05-16 11:20:44 [main ] INFO    [#updates[ 200] train loss[8.22711] remaining[0:14:33]]
2024-05-16 11:21:18 [main ] INFO    [#updates[ 300] train loss[7.78528] remaining[0:13:58]]
2024-05-16 11:21:52 [main ] INFO    [#updates[ 400] train loss[7.50866] remaining[0:13:22]]
2024-05-16 11:22:26 [main ] INFO    [#updates[ 500] train loss[7.30894] remaining[0:12:47]]
2024-05-16 11:23:01 [main ] INFO    [#updates[ 600] train loss[7.16450] remaining[0:12:12]]
2024-05-16 11:23:35 [main ] INFO    [#updates[ 700] train loss[7.02247] remaining[0:11:39]]
2024-05-16 11:24:09 [main ] INFO    [#updates[ 800] train loss[6.90500] remaining[0:11:04]]
2024-05-16 11:24:43 [main ] INFO    [#updates[ 900] train loss[6.82426] remaining[0:10:29]]
2024-05-16 11:25:17 [main ] INFO    [#updates[ 1000] train loss[6.72791] remaining[0:09:55]]
2024-05-16 11:25:52 [main ] INFO    [#updates[ 1100] train loss[6.65094] remaining[0:09:20]]
2024-05-16 11:26:26 [main ] INFO    [#updates[ 1200] train loss[6.57609] remaining[0:08:46]]
2024-05-16 11:26:59 [main ] INFO    [#updates[ 1300] train loss[6.50526] remaining[0:08:11]]
2024-05-16 11:27:34 [main ] INFO    [#updates[ 1400] train loss[6.45236] remaining[0:07:37]]
2024-05-16 11:28:09 [main ] INFO    [#updates[ 1500] train loss[6.40265] remaining[0:07:04]]
2024-05-16 11:28:43 [main ] INFO    [#updates[ 1600] train loss[6.35527] remaining[0:06:29]]
2024-05-16 11:29:16 [main ] INFO    [#updates[ 1700] train loss[6.30736] remaining[0:05:55]]
2024-05-16 11:29:51 [main ] INFO    [#updates[ 1800] train loss[6.26809] remaining[0:05:21]]
2024-05-16 11:30:26 [main ] INFO    [#updates[ 1900] train loss[6.22399] remaining[0:04:46]]
2024-05-16 11:31:00 [main ] INFO    [#updates[ 2000] train loss[6.17303] remaining[0:04:12]]
2024-05-16 11:31:34 [main ] INFO    [#updates[ 2100] train loss[6.13298] remaining[0:03:38]]
2024-05-16 11:32:07 [main ] INFO    [#updates[ 2200] train loss[6.08235] remaining[0:03:03]]
2024-05-16 11:32:40 [main ] INFO    [#updates[ 2300] train loss[6.04140] remaining[0:02:29]]
2024-05-16 11:33:14 [main ] INFO    [#updates[ 2400] train loss[5.99597] remaining[0:01:55]]
2024-05-16 11:33:48 [main ] INFO    [#updates[ 2500] train loss[5.95635] remaining[0:01:21]]
2024-05-16 11:34:23 [main ] INFO    [#updates[ 2600] train loss[5.91822] remaining[0:00:47]]
2024-05-16 11:34:57 [main ] INFO    [#updates[ 2700] train loss[5.88483] remaining[0:00:12]]
2024-05-16 11:36:16 [main ] INFO    [scheduler_type ms]
2024-05-16 11:36:20 [main ] INFO    [Saved the new best model and prediction]
2024-05-16 11:36:20 [main ] WARNING [Epoch 0 - dev EM: 55.781 F1: 65.674 (best EM: 55.781 F1: 65.674)]
2024-05-16 11:36:20 [main ] WARNING [At epoch 1]
2024-05-16 11:36:21 [main ] INFO    [#updates[ 2739] train loss[5.87273] remaining[0:20:50]]
2024-05-16 11:45:47 [main ] INFO    [#updates[ 4400] train loss[5.35978] remaining[0:06:06]]

```

图 11. 模型训练结果示例 2

训练过程将保存在“san.txt”中，根据“san.txt”中的每一轮得到的 loss 值以及两个评估指标 EM 和 F1，使用 python 进行绘图。



图 12. 模型训练的损失值随更新的变化图

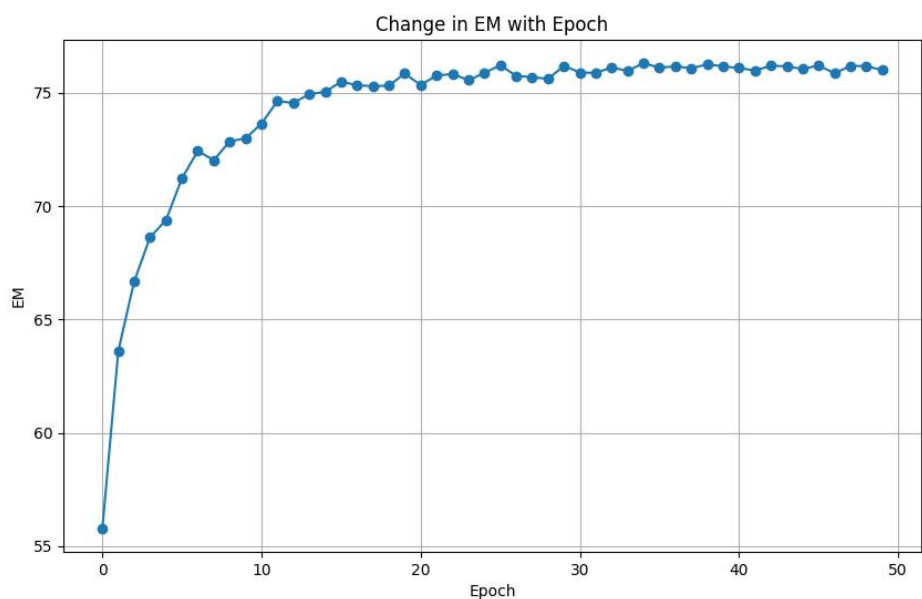


图 13. 评估指标 EM 随 Epoch 的变化图

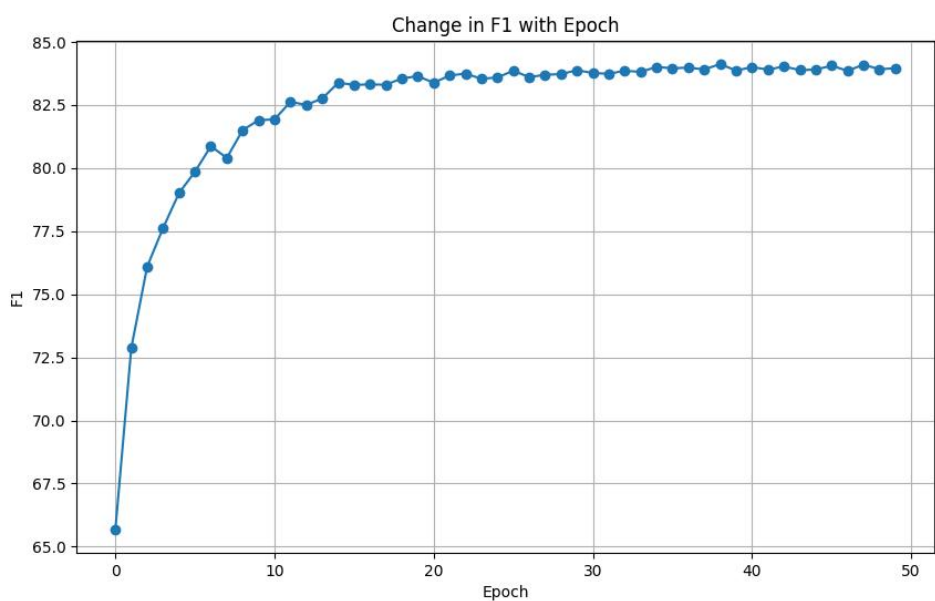


图 14. 评估指标 F1 随 Epoch 的变化图

五、 结果分析

从图 12 可以看出，随着时间的向前推移，训练的损失值越来越小，从刚开始的 10.12 下降到最后的 2.32，可以看到随着训练的进行，模型在训练集上的性能不断提高。从图 13 可以看出，随着 epoch 的增加，评估指标 F1 成上升趋势，当 epoch 大于 30 开始，EM 值在 76.00 上下波动，可以看出随着 epoch 的增加，

模型的预测结果与真实结果匹配得越好。从图 14 可以看出，随着 epoch 的增加，评估指标 EM 成上升趋势，当 epoch 大于 30 开始，EM 值在 84.00 上下波动，可以看出随着 epoch 的增加，模型的性能越好。通过三个参数综合来看，模型的性能随着训练轮数的增加逐渐变化。

六、 心得体会

通过本次实验，我对机器阅读理解系统有了一定的了解，通过研读代码以及对应的论文，对模型的架构有了清晰的认识。

在这个实验中，我实现了一个机器阅读理解模型，同时还使用自动测评指标对模型的翻译效果进行了评测。在代码实现过程中，首先我需要对数据集进行预处理，构建问题和文章的不同特征。在开始训练模型之前，深入理解数据集是至关重要的。在这个实验中，使用的数据集为 SQuAD v1.1，这是我第一次接触着个数据集，通过查阅资料对该数据集有了一定的了解。

然后，在 SQuAD v1.1 上训练 SAN 模型，最后通过两个评估指标，分别为 EM 和 F1，来评估模型的好坏。在这个过程中，我对这两个评估指标有了更加深入的认识，EM 值越高，表示模型的预测结果与真实结果匹配得越好；F1 分数的值越高表示模型的性能越好。通过这两个值，还可以对模型进行适当的优化，调节参数，得到更优的模型。

运行 SAN-MRC 模型的实验需要进行数据预处理、训练模型以及评估模型，并进行深入的结果分析，不断优化模型性能。通过将理论上的知识，自己动手进行实践，跑通整个模型，让我对模型的了解更加深入，增加了我在自然语言处理方面的经验。通过，了解模型结构也增加了我对 NLP 的兴趣，想要更加深入去了解。